1. Write a Python code to implement Caesar Cipher

```
def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            shift_base = 65 if char.isupper() else 97
            result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

message = "Hello World"
shift = 3

encrypted = caesar_encrypt(message, shift)
decrypted = caesar_decrypt(encrypted, shift)

print("Original:", message)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Output:-

Original: Hello World

Encrypted: Khoor Zruog

Decrypted: Hello World

2. Write a Python code to implement Transposition Cipher

```
def encrypt(message, key):
    cipher = [''] * key
    for col in range(key):
        pointer = col
        while pointer < len(message):
            cipher[col] += message[pointer]
            pointer += key
    return ''.join(cipher)

def decrypt(cipher, key):
    num_cols = len(cipher) // key + (len(cipher) % key != 0)
    num_rows = key
    num_shaded = num_cols * num_rows - len(cipher)

    plain = [''] * num_cols
    col = row = 0

    for symbol in cipher:
        plain[col] += symbol
        col += 1
        if col == num_cols or (col == num_cols - 1 and row >= num_rows - num_shaded):
            col = 0
            row += 1
```

```
    return ''.join(plain)


msg = "HELLO WORLD"
key = 4


enc = encrypt(msg, key)
dec = decrypt(enc, key)


print("Original Text:", msg)
print("Key:", key)
print("Encrypted Text:", enc)
print("Decrypted Text:", dec)
```

Output:-

Original Text: HELLO WORLD

Key: 4

Encrypted Text: HORE LLWDLO

Decrypted Text: HELLO WORLD

3. Write a Python code to implement Rail Fence Cipher

```
def encrypt(text, key):
    rail = [['\n' for _ in range(len(text))] for _ in range(key)]
    dir_down = False
    row, col = 0, 0

    for char in text:
        if row == 0 or row == key - 1:
            dir_down = not dir_down
        rail[row][col] = char
        col += 1
        row += 1 if dir_down else -1

    result = []
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result.append(rail[i][j])
    return "".join(result)

def decrypt(cipher, key):
    rail = [['\n' for _ in range(len(cipher))] for _ in range(key)]
    dir_down = None
    row, col = 0, 0
```

```
for i in range(len(cipher)):
    if row == 0:
        dir_down = True
    if row == key - 1:
        dir_down = False
    rail[row][col] = '*'
    col += 1
    row += 1 if dir_down else -1

index = 0
for i in range(key):
    for j in range(len(cipher)):
        if rail[i][j] == '*' and index < len(cipher):
            rail[i][j] = cipher[index]
            index += 1

result = []
row, col = 0, 0
for i in range(len(cipher)):
    if row == 0:
        dir_down = True
    if row == key - 1:
        dir_down = False
    result.append(rail[row][col])
    col += 1
    row += 1 if dir_down else -1

return "".join(result)
```

```
msg = "HELLO WORLD"

key = 3


enc = encrypt(msg, key)

dec = decrypt(enc, key)


print("Original Text:", msg)

print("Key:", key)

print("Encrypted Text:", enc)

print("Decrypted Text:", dec)
```

Output:-

Original Text: HELLO WORLD

Key: 3

Encrypted Text: HOREL OLLWD

Decrypted Text: HELLO WORLD

4. Write a Python code to implement Vigenere Cipher

```
def generate_key(text, key):
    key = list(key)
    for i in range(len(text) - len(key)):
        key.append(key[i % len(key)])
    return "".join(key)

def encrypt(text, key):
    result = ""
    key = generate_key(text, key)
    for i in range(len(text)):
        if text[i].isalpha():
            shift = 65 if text[i].isupper() else 97
            result += chr((ord(text[i]) + ord(key[i]) - 2*shift) % 26 + shift)
        else:
            result += text[i]
    return result

def decrypt(text, key):
    result = ""
    key = generate_key(text, key)
    for i in range(len(text)):
        if text[i].isalpha():
            shift = 65 if text[i].isupper() else 97
            result += chr((ord(text[i]) - ord(key[i])) % 26 + shift)
```

```
        else:
            result += text[i]
    return result

msg = "HELLO WORLD"
key = "KEY"

enc = encrypt(msg, key)
dec = decrypt(enc, key)

print("Original Text:", msg)
print("Key:", key)
print("Encrypted Text:", enc)
print("Decrypted Text:", dec)
```

Output:-

```
Original Text: HELLO WORLD
Key: KEY
Encrypted Text: RIJVS GSPVH
Decrypted Text: HELLO WORLD
```

5. Write a Python code to implement DES (Data Encryption Standard)

```
# pip install pycryptodome

from Crypto.Cipher import DES

from Crypto.Util.Padding import pad, unpad


text = b'HELLO123'

key = b'8bytekey'


cipher = DES.new(key, DES.MODE_ECB)


enc = cipher.encrypt(pad(text, 8))

dec = unpad(cipher.decrypt(enc), 8)


print("Original Text:", text)

print("Key:", key)

print("Encrypted Text:", enc)

print("Decrypted Text:", dec)
```

Output:-

```
Original Text: b'HELLO123'

Key: b'8bytekey'

Encrypted Text: b'E\xba\xa1\x11k\x82\xa6\xab\x97F \xab\xef^\xf2J'

Decrypted Text: b'HELLO123'
```

6. Write a Python code to implement Diffie-Hellman Key Exchange

```
p = 23

g = 5


a = 6

b = 15


A = pow(g, a, p)

B = pow(g, b, p)


key1 = pow(B, a, p)

key2 = pow(A, b, p)


print("Public Values: p =", p, "g =", g)

print("Private Keys: a =", a, "b =", b)

print("Shared Key User1:", key1)

print("Shared Key User2:", key2)
```

Output:-

```
Public Values: p = 23 g = 5

Private Keys: a = 6 b = 15

Shared Key User1: 2

Shared Key User2: 2
```

7. Write a Python code to implement RSA Algorithm

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def modinv(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d

p = 3
q = 11
n = p * q
phi = (p-1)*(q-1)

e = 3
d = modinv(e, phi)

msg = 7

enc = pow(msg, e, n)
dec = pow(enc, d, n)
```

```
print("Original Message:", msg)
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
print("Encrypted Message:", enc)
print("Decrypted Message:", dec)
```

Output:-

Original Message: 7

Public Key (e, n): (3, 33)

Private Key (d, n): (7, 33)

Encrypted Message: 13

Decrypted Message: 7

8. Write a Python code to implement Digital Signature

```
import hashlib

def sign(message, key):
    hash_val = hashlib.sha256(message.encode()).hexdigest()
    return hash_val + key


def verify(message, signature, key):
    hash_val = hashlib.sha256(message.encode()).hexdigest()
    return signature == hash_val + key


msg = "HELLO"
key = "secret"
sig = sign(msg, key)


print("Original Message:", msg)
print("Key:", key)
print("Generated Signature:", sig)
print("Verification Result:", verify(msg, sig, key))
```

Output:-

Original Message: HELLO

Key: secret

Generated Signature:
3733cd977ff8eb18b987357e22ced99f46097f31ecb239e878ae63760e83e4d5secret

Verification Result: True